🎯CSS Animations - Complete Notes

📌 Introduction - CSS Animations Kya Hain?

CSS Animations ka use karke hum elements ko **smoothly move, change ya transform** kar sakte hain bina JavaScript ke. Matlab, agar aapko kisi button ko hover pe bada karna hai, ya koi loader spin karwana hai, ya text ko slide-in karwana hai — yeh sab pure CSS se possible hai.

Real life example: Jaise Instagram ka loading spinner, ya kisi website ka "Welcome" text jo upar se neeche aata hai — yeh sab CSS animations hain.

🆚 Transition vs Animation - Difference Samjho

Bahut students confuse ho jaate hain in dono mein. Dhyan se padho:

Transition

- **Trigger** chahiye (jaise `:hover`, `:focus`, ya class change)
- Sirf **start** aur **end** state hoti hai
- Ek hi baar chalti hai
- Simple effects ke liye best

Animation

- **Khud chalti hai**, koi trigger nahi chahiye
- Multiple steps ho sakte hain (`@keyframes` se)
- Loop kar sakti hai (infinite bhi)
- Complex effects ke liye perfect

Yaad rakhne ka tarika: Transition = "A se B". Animation = "A se B se C se D...".

🎬 @keyframes - Animation Ki Jaan

`@keyframes` rule mein hum **define karte hain** ki animation ke dauraan element kaise change hoga. Yeh ek timeline ki tarah hai jisme aap bata sakte ho ki kis time pe kya hona chahiye.

Basic Syntax:

```
@keyframes animationName {
  from {
    /* starting styles */
  }
  to {
    /* ending styles */
  }
}
```

Ya phir percentage ke saath:

```
@keyframes slideIn {
  0% {
    transform: translateX(-100%);
    opacity: 0;
  }
  50% {
    opacity: 0.5;
  }
  100% {
    transform: translateX(0);
    opacity: 1;
  }
}
```

Note: `from` = `0%` aur `to` = `100%`. Dono same cheez hain, bas alag syntax hai.

⚙️ Animation Properties - Ek Ek Karke

Ab dekhte hain wo saari properties jo animation ko control karti hain:

1️⃣ `animation-name`

Yeh batata hai konsi `@keyframes` use karni hai.

```
.box {
  animation-name: slideIn;
}
```

2️⃣ `animation-duration`

Animation kitni der chalegi. Seconds (`s`) ya milliseconds (`ms`) mein.

```
.box {
  animation-duration: 2s;       /* 2 seconds */
  animation-duration: 500ms;    /* 0.5 seconds */
}
```

Important: Duration zaroor dena hota hai, warna animation nahi chalegi (default `0s` hai).

3️⃣ `animation-timing-function`

Animation ki **speed curve** decide karta hai. Matlab animation start mein fast hogi ya slow, end mein kaisi hogi.

```
.box {
  animation-timing-function: ease;          /* default - slow
start, fast middle, slow end */
  animation-timing-function: linear;        /* constant speed
*/
  animation-timing-function: ease-in;       /* slow start */
```

```
  animation-timing-function: ease-out;       /* slow end */
  animation-timing-function: ease-in-out;    /* slow start aur
slow end */
  animation-timing-function: cubic-bezier(0.4, 0, 0.2, 1);  /
* custom curve */
  animation-timing-function: steps(5);       /* 5 discrete ste
ps */
}
```

Tip: `linear` use karo jab cheez constantly move kare (jaise loader). `ease-out` natural feel deta hai UI ke liye.

4️⃣ `animation-delay`

Animation start hone se pehle kitna wait karna hai.

```
.box {
  animation-delay: 1s;     /* 1 second wait karke start hogi
*/
  animation-delay: -1s;    /* negative bhi de sakte ho - midd
le se start hogi */
}
```

5️⃣ `animation-iteration-count`

Animation kitni baar chalegi.

```
.box {
  animation-iteration-count: 1;          /* default - ek baar
*/
  animation-iteration-count: 3;          /* teen baar */
  animation-iteration-count: infinite;   /* hamesha chalti ra
hegi */
  animation-iteration-count: 2.5;        /* decimal bhi valid
hai */
}
```

6️⃣ `animation-direction`

Animation kis direction mein chalegi.

```
.box {
  animation-direction: normal;            /* default: 0% se
100% */
  animation-direction: reverse;           /* 100% se 0% */
  animation-direction: alternate;         /* normal, phir re
verse, phir normal... */
  animation-direction: alternate-reverse; /* reverse, phir n
ormal, phir reverse... */
}
```

Real example: Heartbeat effect banane ke liye `alternate` use hota hai.

7️⃣ `animation-fill-mode`

Animation se **pehle** aur **baad** mein element kaisa dikhega — yeh control karta hai.

```
.box {
  animation-fill-mode: none;        /* default - animation ke
pehle/baad original state */
  animation-fill-mode: forwards;    /* animation ke baad 100%
wali state mein ruk jayega */
  animation-fill-mode: backwards;   /* animation start hone s
e pehle 0% wali state */
  animation-fill-mode: both;        /* forwards + backwards d
ono */
}
```

Sabse important property hai yeh. Bahut log shikayat karte hain "animation ke baad element wapis original pe aa jata hai" — solution hai `forwards`.

8️⃣ `animation-play-state`

Animation chalu hai ya paused hai.

```css
.box {
  animation-play-state: running;   /* default - chalu */
  animation-play-state: paused;    /* ruki hui */
}

.box:hover {
  animation-play-state: paused;    /* hover pe pause kar do
*/
}
```

🎯 Shorthand Property - Sab Ek Line Mein

Har property alag-alag likhne ki zaroorat nahi, ek line mein likh sakte ho:

```css
.box {
  animation: name duration timing-function delay iteration-co
unt direction fill-mode play-state;
}
```

Example:

```css
.box {
  animation: slideIn 2s ease-out 0.5s infinite alternate forw
ards;
}
```

Order yaad rakhne ka trick: Naam, kitni der, kaisi speed, delay, kitni baar, direction, fill-mode.

💻 Practical Examples - Code With Explanation

Example 1: Bouncing Ball

```css
.ball {
  width: 50px;
  height: 50px;
  background: red;
  border-radius: 50%;
  animation: bounce 1s ease-in-out infinite alternate;
}

@keyframes bounce {
  from {
    transform: translateY(0);
  }
  to {
    transform: translateY(-100px);
  }
}
```

Yahan ball upar-neeche bounce karegi infinitely.

Example 2: Loading Spinner

```css
.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid #ccc;
  border-top-color: #333;
  border-radius: 50%;
  animation: spin 1s linear infinite;
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
```

Classic loader — har website pe dikhta hai.

Example 3: Fade In Text

```css
.welcome-text {
  animation: fadeIn 2s ease-out forwards;
  opacity: 0;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

Text neeche se upar aata hai aur visible ho jata hai.

Example 4: Pulse Effect (Heartbeat)

```css
.heart {
  animation: pulse 1.5s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% {
    transform: scale(1);
  }
  50% {
    transform: scale(1.2);
```

```
  }
}
```

Notice: `0%` aur `100%` ek saath likh sakte ho agar same style hai.

Example 5: Button Hover Animation

```css
.btn {
  padding: 10px 20px;
  background: blue;
  color: white;
  border: none;
  transition: all 0.3s ease;
}

.btn:hover {
  transform: scale(1.1);
  background: darkblue;
  box-shadow: 0 5px 15px rgba(0,0,0,0.3);
}
```

Yeh transition ka example hai, animation ka nahi — but bahut common hai.

🔄 Transform Property - Animation Ki Best Friend

Animations ke saath `transform` property bahut use hoti hai kyunki yeh **GPU pe chalti hai** (smooth performance).

Common Transforms:

```css
transform: translateX(100px);        /* X axis pe move */
transform: translateY(-50px);        /* Y axis pe move */
transform: translate(50px, 100px);   /* dono ek saath */
transform: scale(1.5);               /* 1.5x bada */
transform: scaleX(2);                /* sirf horizontal */
transform: rotate(45deg);            /* 45 degree ghuma */
```

```
transform: skew(10deg, 20deg);        /* tedha karna */

/* Multiple transforms ek saath */
transform: translateX(50px) rotate(45deg) scale(1.2);
```

Performance Tip: Animation karte time `top`, `left`, `width`, `height` ki jagah `transform` use karo — bahut smooth chalega.

⚡ Performance Best Practices

1. `transform` **aur** `opacity` **ko prefer karo** — yeh GPU-accelerated hain
2. **Layout-changing properties avoid karo** (`width`, `height`, `margin`) animations mein
3. `will-change` **property** use kar sakte ho heavy animations ke liye:

   ```
   .animated-element {
     will-change: transform;
   }
   ```

4. **Bahut zyada animations ek saath mat chalao** — page slow ho jata hai
5. **Mobile pe test karo** — desktop pe smooth lag sakti hai, mobile pe lag karegi

---

🌐 Browser Compatibility

Modern browsers (Chrome, Firefox, Edge, Safari) mein CSS animations almost fully supported hain. Lekin agar aap old browsers ko support karna chahte ho to vendor prefixes use karo:

```
@-webkit-keyframes slideIn { /* ... */ }
@keyframes slideIn { /* ... */ }

.box {
  -webkit-animation: slideIn 2s;
```

```
  animation: slideIn 2s;
}
```

Aaj kal `-webkit-` prefix ki zaroorat kam hi padti hai, modern browsers handle kar lete hain.

🎓 Quick Revision - Cheat Sheet

Property	Kaam
`animation-name`	Konsi `@keyframes` use karni hai
`animation-duration`	Kitni der chalegi
`animation-timing-function`	Speed curve
`animation-delay`	Kitna wait karke start hogi
`animation-iteration-count`	Kitni baar chalegi
`animation-direction`	Direction (normal/reverse/alternate)
`animation-fill-mode`	Pehle aur baad ki state
`animation-play-state`	Running ya paused

📝 Practice Exercises (Homework)

Inhe try karo, samajh aa jayega:

1. **Easy:** Ek square banao jo continuously rotate kare
2. **Easy:** Text banao jo type-writer effect se aaye (hint: `width` aur `steps()` use karo)
3. **Medium:** Loading dots banao — 3 dots jo ek-ek karke bounce karein (delay use karo)
4. **Medium:** Card flip animation banao hover pe
5. **Hard:** Ek complete navigation menu banao jo slide-in ho with staggered animations
6. **Hard:** Solar system banao — sun ke around planets ghumte hue

---

❓ Common Interview Questions

1. CSS animations aur transitions mein kya farak hai?
2. `@keyframes` kya hota hai? Iska syntax samjhao.
3. `animation-fill-mode: forwards` ka use kya hai?
4. Animation ko pause kaise karte hain?
5. CSS animation kaisi properties pe best perform karti hai aur kyun?
6. `cubic-bezier()` kya hai?
7. `transform` ko animations ke saath prefer kyun karte hain?

💡 Pro Tips

- ✅ **Hamesha** `animation-duration` **zaroor do**, warna kuch nahi hoga
- ✅ Complex animations ke liye **percentage keyframes** use karo
- ✅ `prefers-reduced-motion` media query ka use karo accessibility ke liye:

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important;
    transition: none !important;
  }
}
```

- ✅ Animations **subtle** rakho — zyada flashy mat banao, user ko irritation hoti hai
- ✅ **Mobile devices pe test karo** hamesha
- ❌ Bahut saari heavy animations ek saath mat chalao
- ❌ Important content ko purely animation pe dependent mat banao

---

🎯 Summary

CSS Animations 3 cheezon se milke banti hai:

1. `@keyframes` — animation ki definition
2. `animation-*` **properties** — animation ka behavior control
3. `transform` **/** `opacity` — actual visual changes

Bas itna yaad rakho:

> "Pehle `@keyframes` banao, phir element pe `animation` property lagao."

Practice karte raho! Animation ek aisi cheez hai jo padhne se zyada karne se aati hai. Khud try karo, browser mein dekho, aur experiment karte raho. 🚀

Happy Coding! 💻